

Ingenic®

Halley6 Software Platform Rapid development guide

Author: System Software Department

Version: 2.0

Date: 2022.12.2



北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

Ingenic®

Halley6 Software Platform Rapid Development Guide

Copyright © Ingenic Semiconductor Co. Ltd 2022. All rights reserved.

Release history

Date	Revision	Change
2021.10.26	1.0	First release
2022.12.2	2.0	Second release

Disclaimer

This documentation is provided for use with Ingenic products. No license to Ingenic property rights is granted. Ingenic assumes no liability, provides no warranty either expressed or implied relating to the usage, or intellectual property right infringement except as provided for by Ingenic Terms and Conditions of Sale.

Ingenic products are not designed for and should not be used in any medical or life sustaining or supporting equipment.

All information in this document should be treated as preliminary. Ingenic may make changes to this document without notice. Anyone relying on this documentation should contact Ingenic for the current documentation and errata.

Ingenic Semiconductor Co., Ltd.

Ingenic Headquarters, East Bldg. 14, Courtyard #10
Xibeiwang East Road, Haidian District, Beijing, China,
Tel: 86-10-56345000
Fax: 86-10-56345001
Http: [//www.ingenic.com](http://www.ingenic.com)

Catalogue

1	SDK Engineering download.....	1
1.1	SDK download.....	1
1.2	Buildroot download.....	1
1.3	Project Decompression.....	1
2	SDK Introduction.....	3
2.1	Overall structure of the project.....	3
2.2	Document Description.....	4
3	SDK Compile.....	5
3.1	Overall Compilation.....	5
3.2	Separate compilation.....	6
3.2.1	Compile uboot.....	6
3.2.2	Compile kernel.....	6
3.2.3	Compile buildroot.....	6
3.2.4	Compiling the module.....	7
3.2.4.1	Mma command to compile modules individually.....	7
3.2.4.2	Target end module compilation.....	8
3.2.4.3	Target side module clean.....	8
3.2.5	Compile the file system image.....	8
3.2.6	Packaging file system images.....	8
4	Use of burning tools.....	10
4.1	Burn SFC Nand.....	10
4.2	Burn SFC Nor.....	12
4.3	Burn SD card.....	14
4.4	Burn eMMC.....	16
4.5	Erase option of burning tool.....	16
4.6	Other configuration.....	17
4.6.1	Configure serial port.....	17
4.6.2	Secure programming.....	18
4.6.3	SN burn.....	18
5	FAQ.....	19
5.1	How to import the compiled executable program module to the development board?.....	19
5.2	Problems compiling the kernel.....	19
5.3	The program cannot be executed or the link fails at compile time.....	19

Key words	Compile, burn, test
abstract	This document is applicable to the Halley6 development board launched by Junzheng. The purpose is to help developers quickly understand the organization structure, software compilation, program burning, and corresponding functional test demonstration of the software development platform supporting the development board.
abbreviation	SDK project -- project, SDK project top-level directory -- project directory
References	

1 SDK Engineering download

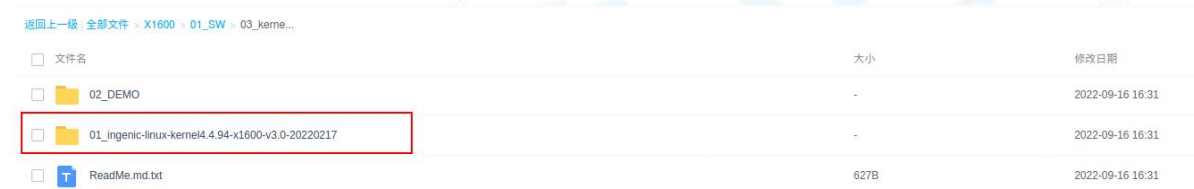
Development board matching SDK source code projects are released through Baidu web disk. This section describes how to download the latest source code project and unzip the associated zip package. Download links are included.

1.1 SDK download

Web disk download link: <https://pan.baidu.com/s/1g5txFp6oEo066aKwA2gIdw>

Extraction code: svtt

Web disk download: development board chip data manual, development board hardware schematic and SDK software package. Here mainly describes the software package download, other information users can choose to download according to their own needs. Select the latest SDK package to download.



返回上一级 全部文件 > X1600 > 01_SW > 03_kerne...	大小	修改日期
☐ 文件名		
☐ 02_DEMO	-	2022-09-16 16:31
☐ 01_ingenic-linux-kernel4.4.94-x1600-v3.0-20220217	-	2022-09-16 16:31
☐ ReadMe.md.txt	627B	2022-09-16 16:31

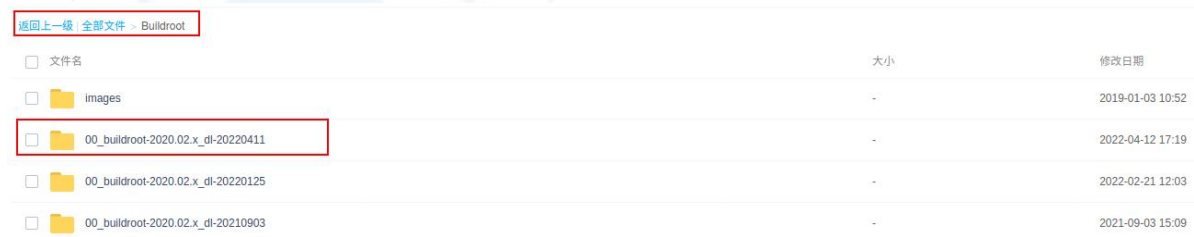
1.2 Buildroot download

The software development platform supporting Halley 5 needs to use Buildroot to provide the most basic file system, so you need to download the Buildroot project that you have modified.

Web disk download link: https://pan.baidu.com/s/1e57Ze_WXoOHGFN6xhcNLkQ

Extraction code: gx42

Also select the latest project download here. *(It is particularly important to note that the version number of dl xxxx must correspond to the SDK version number. In principle, it is recommended to download the latest)*



返回上一级 全部文件 > Buildroot	大小	修改日期
☐ 文件名		
☐ images	-	2019-01-03 10:52
☐ 00_buildroot-2020.02_x_dl-20220411	-	2022-04-12 17:19
☐ 00_buildroot-2020.02_x_dl-20220125	-	2022-02-21 12:03
☐ 00_buildroot-2020.02_x_dl-20210903	-	2021-09-03 15:09

Figure 1-2

1.3 Project Decompression

1、Copy the ingenic-linux-kernel4.4.94-xx.tar.bz2 file from the downloaded SDK project directory to a new directory.

2、Copy dl.tar.bz2 under the downloaded Buildroot project directory to the newly created

directory above. In SDK projects, there is usually a Buildroot project. If there is no need to download and decompress separately.

2、Execute the command to decompress

```
$ tar -xf ingenic-linux-xx.tar.bz2  
$ tar -xf dl.tar.bz2 -C ./buildroot/
```

2 SDK Introduction

2.1 Overall structure of the project

Halley6's supporting software development platform is a Linux system release and development platform developed by Junzheng. The platform's compilation system can support c files and cpp files. At the same time, it can support the compilation and generation of static libraries, dynamic libraries and executable files. The software development platform can also quickly and conveniently integrate and add third-party applications and library files. The project supports overall compilation, separate compilation of modules, and selective compilation. The project directory of this software development platform is shown in Figure 2-1.

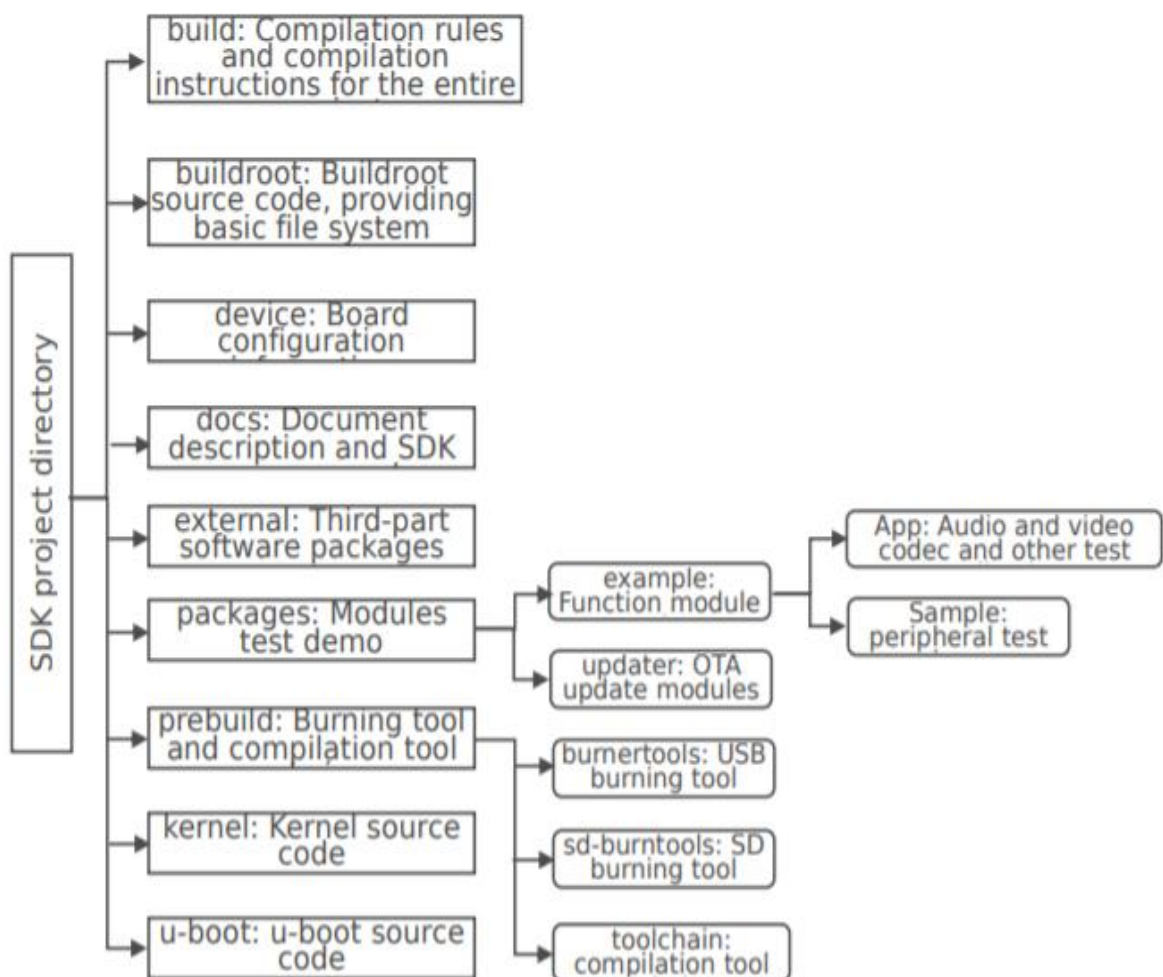


Figure 2-1

2.2 Document Description

The purpose of this document is to help developers quickly understand how to compile SDK and burn programs. If users need to have a deeper understanding of the various peripheral modules and application development of Halley6, they can refer to other documents released with Halley6. Please refer to Table 1 for the description of each document.

Table 1

Document name	Document Content Description
1- Halley6 Software Platform Rapid Development Guide	Help developers quickly understand the compilation, burning and testing of platform software
2- Halley6 uboot Development Manual	U-boot configuration compilation, kernel loading, and file system mounting
3- Halley6 Kernel Development Manual	Each kernel driver module configuration compilation, as well as the corresponding driver source in the kernel structure of the location
4- Halley6 Platform Application Development Manual	Buildroot configuration compilation, user application adding and compiling, application debugging methods
5- USBCloner Quick Start guide to burning tools	USB burn tool in the ubuntu and Windows burn demonstration, the document is located in the project directory/prebuilts/burnertools/Doc_Chinese
6- USBClonerBurning tool description document	USB Slim Use Instructions /prebuilts/burnertools/Doc_Chinese

3 SDK Compile

3.1 Overall Compilation

This section describes how to quickly compile the entire SDK project. The compilation procedure is as follows:

1、Setting environment Variables

Execute the command in the project directory:

```
$ source build/envsetup.sh
```

2、Select device

Execute the command in the project directory:

```
$ lunch
```

After the command is executed, different configuration schemes will be displayed. Please enter the number according to the configuration of the development board, such as Halley6.v10. At the same time, pay attention to distinguishing the storage types of development boards, such as nand and nor. Please refer to Table 2 for the explanation of various options in the lunch configuration. Select different storage types and version numbers after the LUNCH. Configuration demonstration

Table 2

Configuration Option Name	Configuration Description
halley6.v10_nand_4.4.94-eng	The nand of kernel 4.4.94 for compiling v10 development boards starts
halley6.v10_nor_4.4.94-eng	The nor of kernel 4.4.94 for compiling v10 development boards starts
halley6.v10_msc_4.4.94-eng	The msc of kernel 4.4.94 for compiling v10 development boards starts
halley6.v10_nand_5.10-eng	The nand of kernel 5.10 for compiling v10 development boards starts
halley6.v10_nor_5.10-eng	The nor of kernel 5.10 for compiling v10 development boards starts
halley6.v10_msc_5.10-eng	The msc of kernel 5.10 for compiling v10 development boards starts

3、Engineering compilation

make or make MAKE_JLEVEL=8 or make -j8 (multithreaded compilation) in the project directory

```
$ make -j8
```

Note: Due to different machine performance, the compilation time will vary, usually more than 2 hours

At this point, the overall compilation of the project is complete. The output of the compilation result is as follows:

① Project directory /out/product/halley6.v10_nand_4.4.94-eng/image to generate kernel, system.ubifs, and uboot images.

② Project directory /out/product/halley6.v10_nand_4.4.94-eng/obj is an intermediate file output during the compilation of buildroot, kernel, uboot, and packages.

2、Engineering clean

```
$ make clean
```

3.2 Separate compilation

This section mainly describes how to compile uboot, kernel, buildroot and modules separately, and finally describes how to package file system images. Before compiling the module separately, you need to set the environment variables and select the device. Please refer to the relevant description in 2.1 for specific operations. If the current terminal has been set, you do not need to reset it. (*It should be noted that the precondition for separate compilation is that the project has been compiled as a whole*)

3.2.1 Compile uboot

1、Clear the method of uboot compilation results, and execute the command in the project directory:

```
$ make uboot-clean
```

2、Compile uboot and execute the command in the project directory:

```
$ make uboot
```

3.2.2 Compile kernel

1、The method to clear the kernel compilation results, and execute the command in the project directory:

```
$ make kernel-clean
```

2、To configure the kernel, execute the command in the project directory:

```
$ make kernel-menuconfig
```

3、Compile the kernel and execute the command in the project directory:

```
$ make kernel -j8
```

3.2.3 Compile buildroot

1、To clear the method of buildroot to compile intermediate files, execute the command in the project directory:

```
$ make buildroot-clean
```

```
$ make buildroot-distclean
```

- 2、 To configure the buildroot method, execute the command in the project directory:

```
$ make buildroot-menuconfig
```

- 3、 Compile the buildroot method and execute the command in the project directory:

```
$ make buildroot -j8
```

- 4、 The method of compiling the buildroot after partially modifying the buildroot, and executing the command in the project directory:

```
$ make buildroot-rebuild
```

- 5、 After making modifications to some third-party libraries or kernel tool software such as opencv and i2c-tools_4.1, it can be compiled separately by executing the built-in command of buildroot.

```
$ make buildroot-opencv-rebuild
```

```
$ make buildroot-i2c-tools-rebuild
```

The third-party software included in the buildroot project is located in the buildroot/packages directory, and the compilation output is located in /out/product/halley6.v10_nor_4.4.94-eng/obj/buildroot-intermediate/build.

- 6、 The support of other commands of buildroot can be queried through the help option, and the command can be executed in the project directory:

```
$ make buildroot-help
```

3.2.4 Compiling the module

This project supports the compilation of a single module and the clean operation of a single module. After the compiling system of the project is initialized, there are two sets of single-module compiling methods, one is the general module compiling method `mma` command, and the other is using the `make` command directly in the project directory. Before compiling the module separately, you need to ensure that the environment variables of the current terminal have been set. For details, please refer to the relevant descriptions in Section 2.1.

3.2.4.1 Mma command to compile modules individually

The `mma` command is a general single-module compilation method, and does not distinguish whether the module is the host side or the target side. The specific usage of this command is:

- 1、 Go to the module directory to be compiled, the module directory contains the `Build.mk` file.
- 2、 Execute the `mma` command, this module and the modules that this module depends

on will be compiled.

3.2.4.2 Target end module compilation

The target side module compilation is carried out in the project directory, and the command format is:

```
$ make $(LOCAL_MODULE)
```

Command usage example, take the project directory /packages/example/App /cimutils module as an example, the LOCAL_MODULE value of the module in the Build.mk file in its directory is: cimutils. To compile a module separately, you only need to execute the command in the project directory:

```
$ make cimutils
```

The compiled executable file will eventually be installed directly into the corresponding path of the file system, and the corresponding executable file can be found according to the final installation path after compilation.

3.2.4.3 Target side module clean

Target-side module clean can be performed in the project directory, the command format is:

```
$ make $(LOCAL_MODULE)-clean
```

Command usage example, take the project directory /packages/example/App /cimutils module as an example, the LOCAL_MODULE value of the module in the Build.mk file in its directory is: cimutils. A single clean module only needs to execute the command in the project directory:

```
$ make cimutils-clean
```

3.2.5 Compile the file system image

The method of compiling the file system image separately is to execute the command in the project directory:

```
$ make systemimage
```

After the command is executed, the system image will be regenerated in the project directory /out/product/halley6.v10_nor_4.4.94-eng/image.

3.2.6 Packaging file system images

Package the file system image, and package the compiled and output files into the final file system image for burning to the development board. To package a file system image, you need to execute the command in the project directory:

```
$ make post-image
```

The final file system image output is in the project directory
/out/product/halley6.v10_nor_4.4.94-eng/image directory.



4 Use of burning tools

This section mainly describes the use of the USB burning tool that is supported by this development platform. This USB burning tool can be used in ubuntu12.04, windowsXP and Windows7 and above, both 32-bit and 64-bit. This development guide is only for the corresponding programming demonstration using the USB programming tool under ubuntu. Run get-burner in the project directory to download the latest burning tool .Enter the programming tool directory and enter the command to decompress the programming tool file:

```
$ tar -xf cloner- "Version No" -ubuntu_alpha.tar.gz
```

After the decompression is completed, enter the corresponding programming tool directory, and prepare to burn uboot, kernel, and rootfs. Before starting programming, you need to ensure that the two Type-C ports of the development board are connected to the PC.

Note: The "version number" of the actual burning tool is subject to the release of the SDK.

The supported programming configurations are shown in the table below,

Burn Configuration	explain
x1600_sfc_nand_lpddr2_linux.cfg	➤ Nand Start the default configuration
X1600_sfc_nor_lpddr2_linux.cfg	➤ Nor Start the default burning configuration

4.1 Burn SFC Nand

The specific way to program SFC Nand is:

- 1、Enter the programming tool directory and execute the command:

```
$ ./cloner
```

- 2、After the command is executed, a graphical interface software will be opened. First, perform the corresponding configuration and click "Configure".

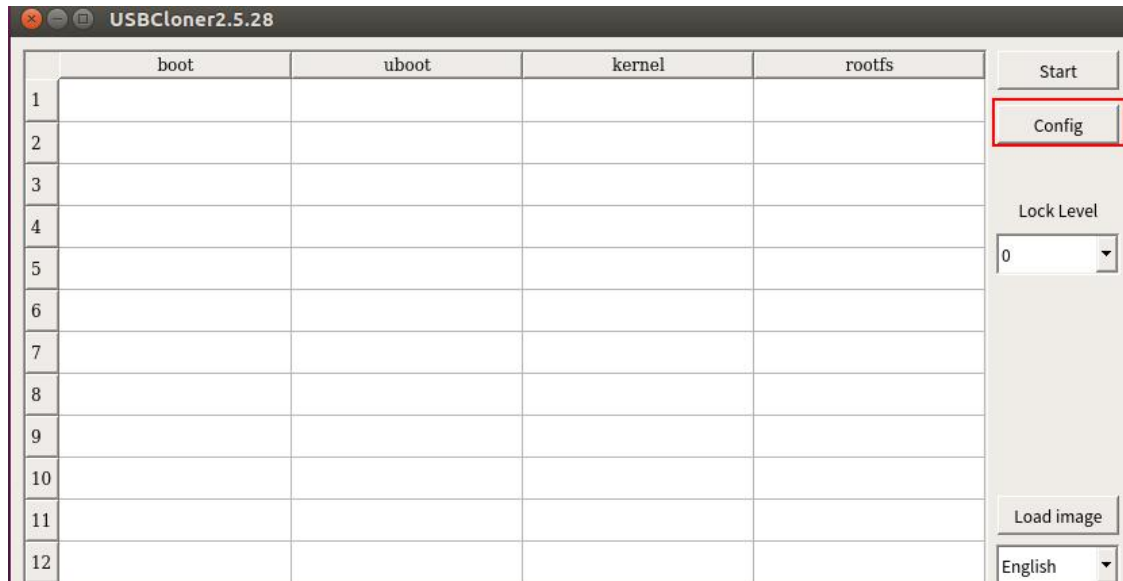


Figure 4-1

3、For the configuration of basic information, click INFO to enter the basic configuration information, select x1600 for the platform and x1600_sfc_nand_lpddr2_linux.cfg for the board level, please refer to Figure 4-2 for details.

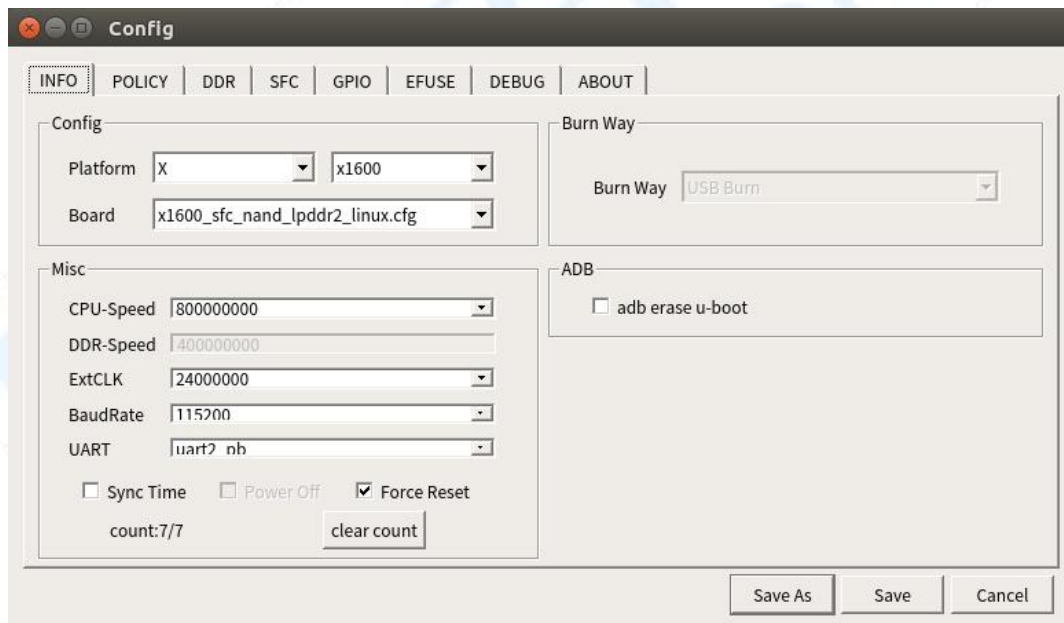


Table 4-2

4、Configure the image file path, click POLICY to configure accordingly. Set the corresponding paths of the uboot, kernel and rootfs image files in sequence (the file names are uboot, kernel, system.ubifs respectively), and click the corresponding setting to select the file. For configuration, please refer to Figure 4-3.

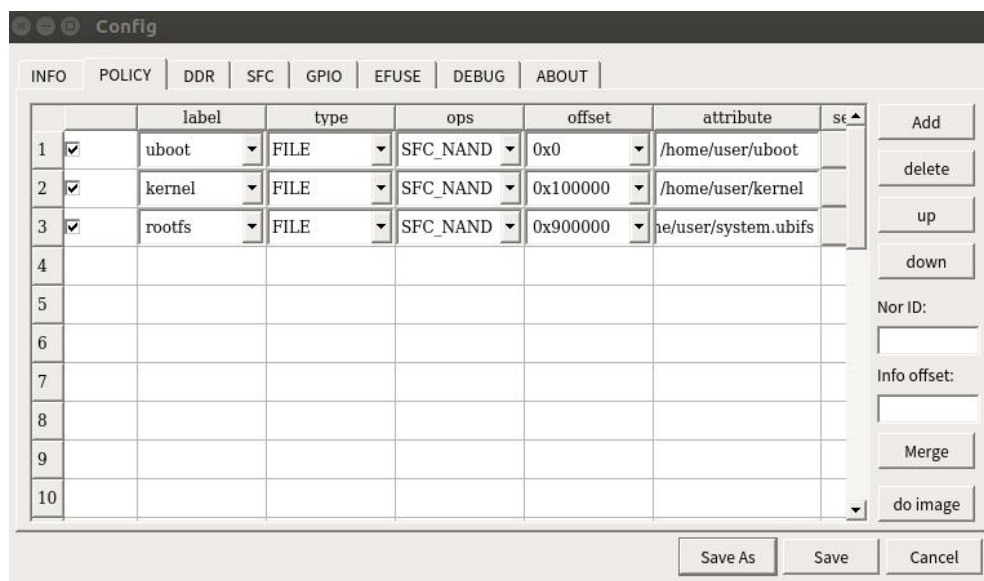


Figure 4-3

- 5、After the setting is completed, click the "Save" button to prepare for programming.
- 6、Click the "Start" button on the main interface of the burning tool.
- 7、The development board enters the programming state, first press the BOOT_SEL1 button of the development board, then press the RST_N button, and after the blue indicator lights up, release the RST_N and BOOT_SEL1 buttons respectively.
- 8、Wait for the burning tool to complete the burning to 100%, as shown in Figure 4-4.

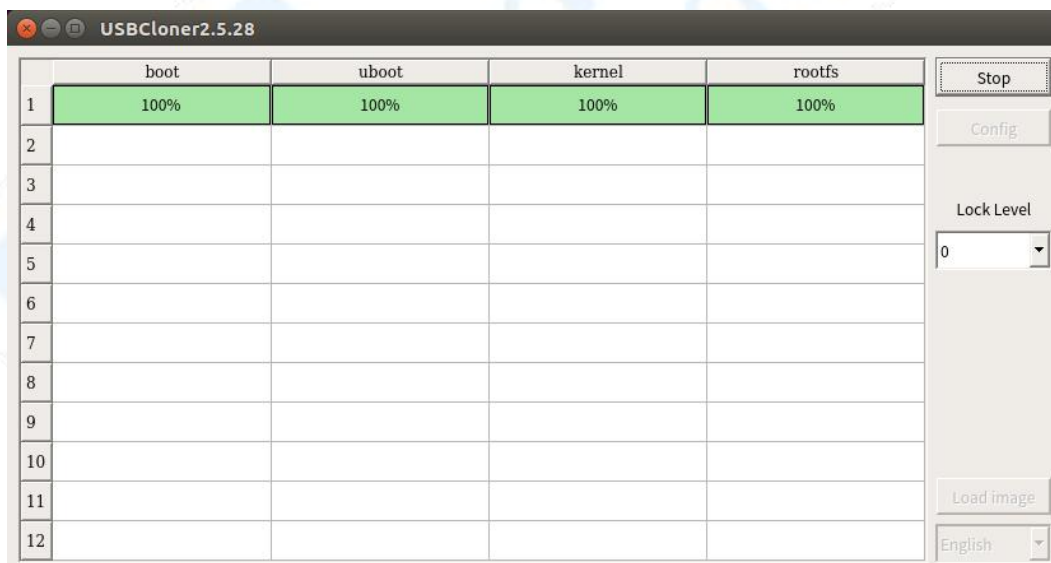


Figure 4-4

4.2 Burn SFC Nor

The difference between programming SFC Nor and SFC Nand lies in the different configurations. The specific configuration methods are:

- 1、For the configuration of basic information, click INFO to enter the basic configuration information, select x1600 for the platform and `x1600_sfc_nor_lpddr2_linux.cfg` for the board level, see Figure 4-5 for details.

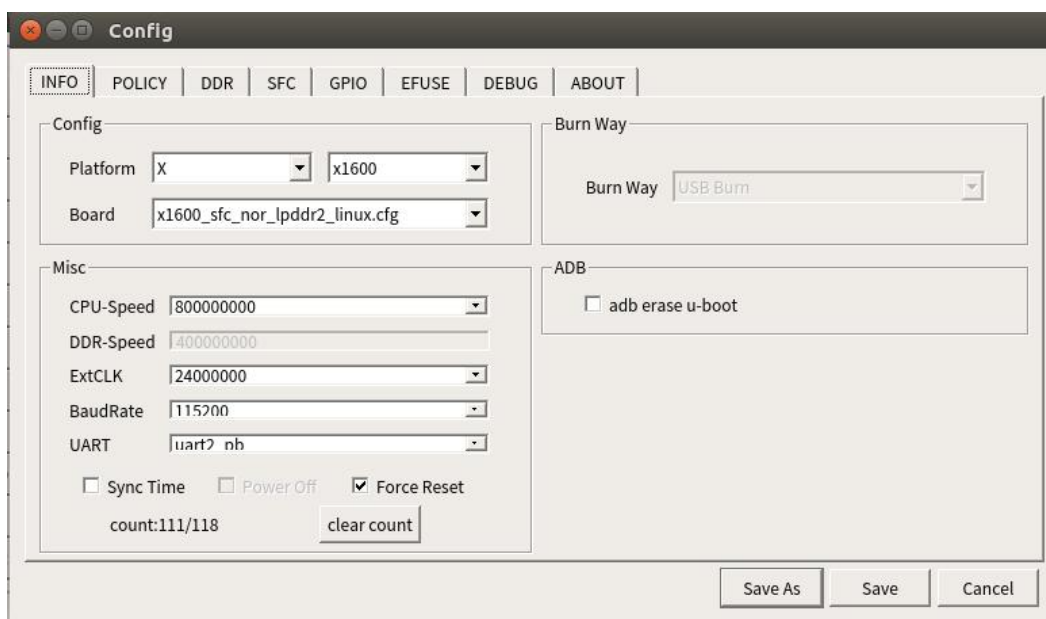


Figure 4-5

2、Configure the image file path, and click POLICY to configure accordingly. Set the corresponding paths of the uboot, kernel and rootfs image files in sequence, and click the corresponding setting to select the file. The configuration can refer to Figure 4-6.

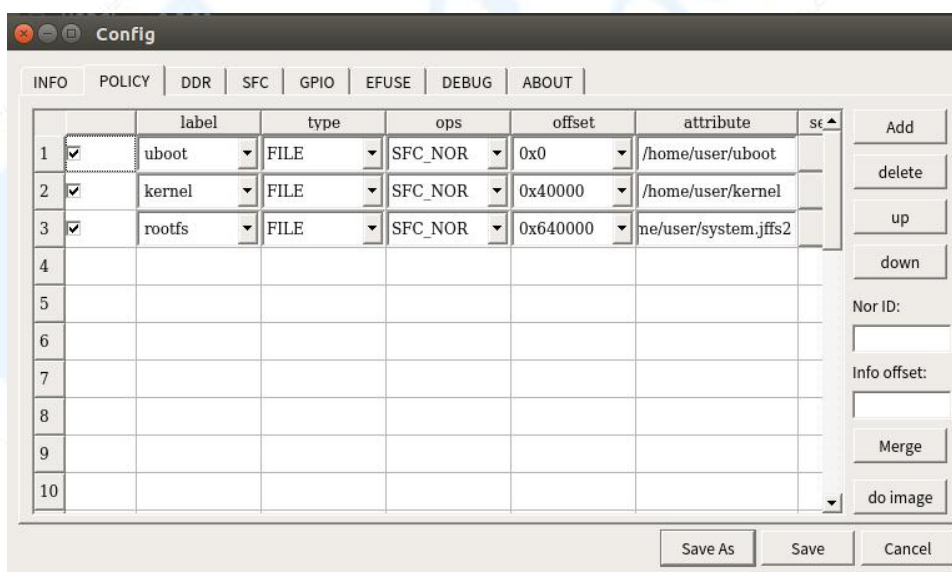


Figure 4-6

3、For SFC configuration, since there are many types of flash supported by the programming tool, it is necessary to solve the problem of ID conflict according to the model of the onboard nor flash before programming. Click norinfo, find the ID of the corresponding onboard nor flash according to the name, and browse whether there is a conflict marked in red. If there is a conflict, choose to delete the others. Refer to Figure 4-7 for the configuration interface.

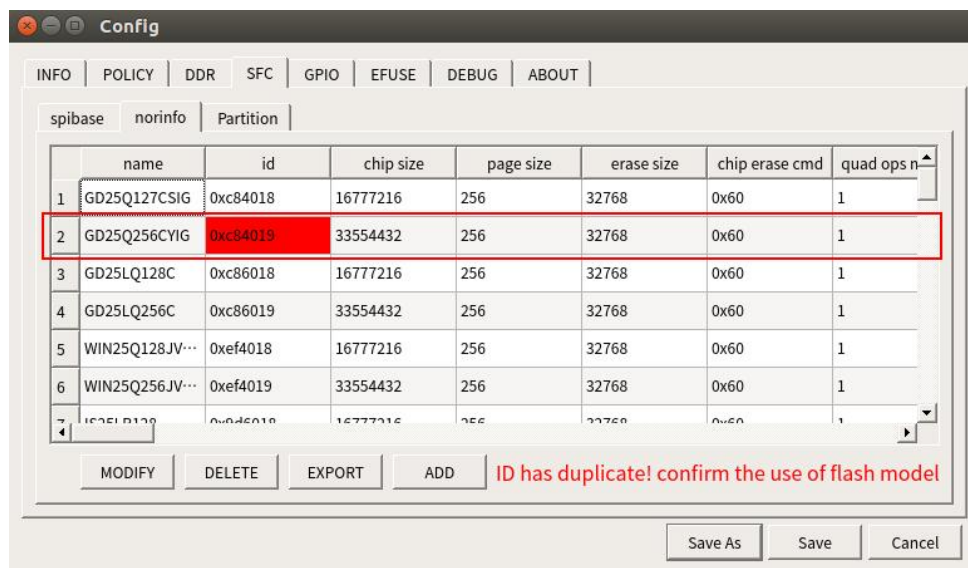


Figure 4-7

- 4、After the setting is completed, click the "Save" button to prepare for programming.
- 5、Click the start button on the main interface of the burning tool.
- 6、The development board enters the programming state, first press the BOOT_SEL1 button of the development board, then press the RST_N button, and after the blue indicator lights up, release the RST_N and BOOTL_SEL1 buttons respectively.
- 7、Wait for the burning tool to burn to 100%, as shown in Figure 4-4

4.3 Burn SD card

This section mainly describes how to burn the image file to the SD card through the USB burning tool, and how to boot from the SD card. The difference between burning SD card and burning Nand mainly lies in the difference in configuration. The specific configuration is:

- 1、For the configuration of basic information, click INFO to enter the basic configuration information, select x1600 for the platform and **x1600_mmc_lpddr2_linux.cfg** for the board level, see Figure 4-8 for details.

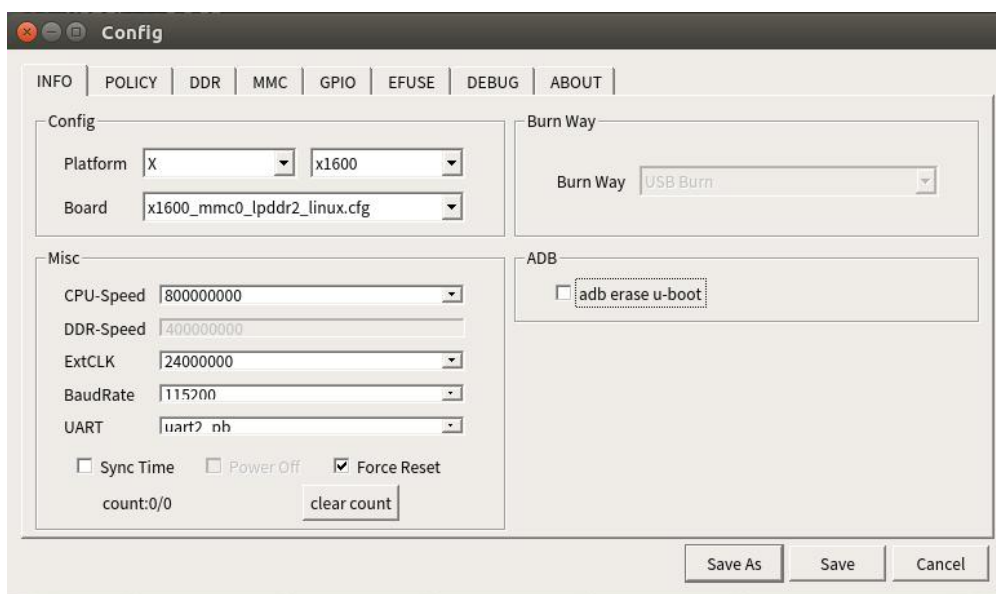


Figure 4-8

2. Configure the image file path and click POLICY to configure accordingly. Set the corresponding paths of the uboot, kernel and rootfs image files in sequence (the file names are uboot, kernel, system.ext2 respectively), and click the corresponding setting to select the file. The configuration can refer to Figure 4-9. It should be noted that the file system formats used by different storage media are different, and all of them need to be re-configured and re-compiled before re-compiling.

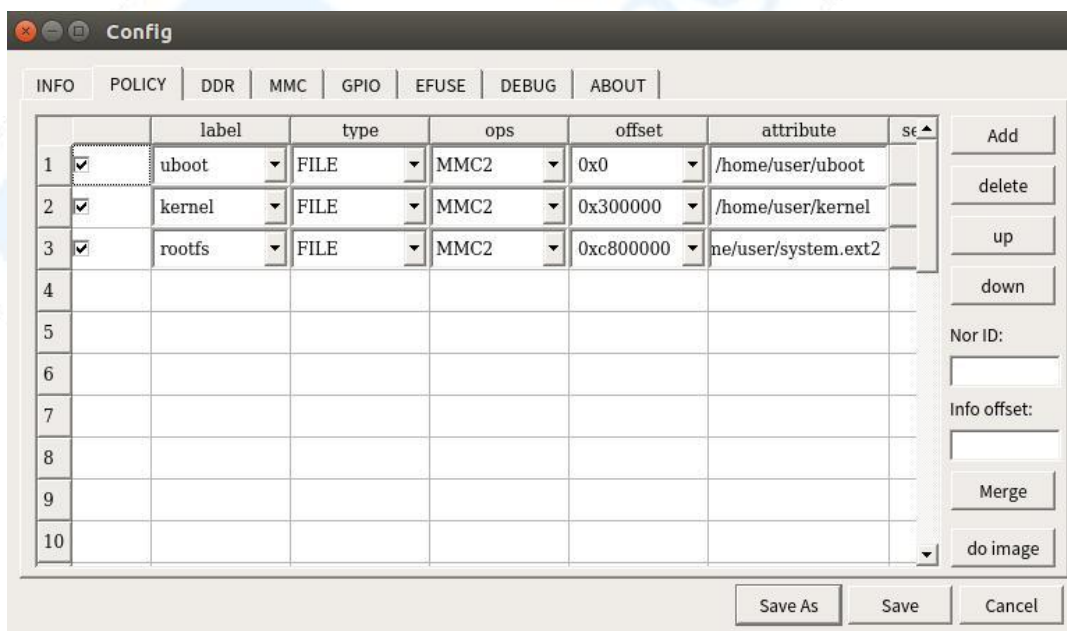


Figure 4-9

3. MMC option configuration, as shown in Figure 4-10.



Figure 4-10

- 4、After the setting is complete, click the save button to prepare for programming.
- 5、Click the "Start" button on the main interface of the burning tool.
- 6、The development board enters the programming state, first press the BOOT_SEL1 button of the development board, then press the RST_N button, and after the blue indicator lights up, release the RST_N and BOOTL_SEL1 buttons respectively.
- 7、Wait for the burning tool to burn to 100%, as shown in Figure 4-4.
- 8、Start from the SD card after the burning is completed: first press the BOOT_SEL1 button of the development board, then press the RST_N button, and after the blue indicator lights up, release the RST_N and BOOTL_SEL1 buttons respectively.

4.4 Burn eMMC

Not currently supported.

4.5 Erase option of burning tool

The erasing options of this burning tool include mass erasing and non-mass erasing. When programming for the first time, you need to select mass erasing. If you only want to burn a certain file in the future, you can remove this option. The related configuration interface is shown in Figure 4-11.

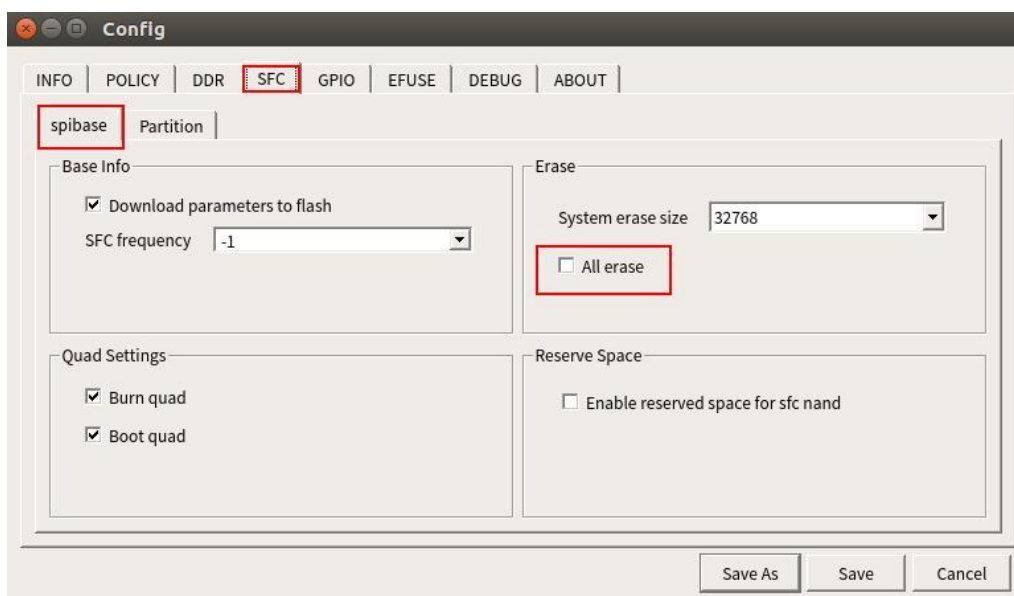


Figure 4-11

4.6 Other configuration

4.6.1 Configure serial port

Serial port is one of the most important tools in the development process. Before using serial port debugging, you need to configure the serial port environment:

- 1、 Install the serial port tool in ubuntu and execute the command:

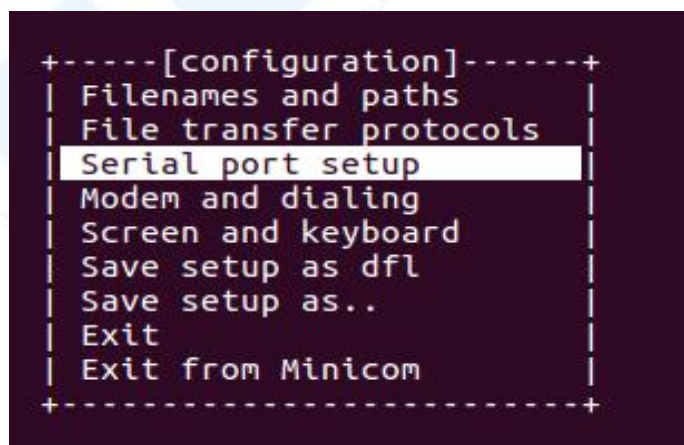
```
$ sudo apt-get install minicom
```

- 2、 Serial port tool configuration

minicom configuration, and then enter the command on the PC terminal:

```
$ sudo minicom -s
```

After the command is executed, it will enter the following interface:



Select Serial port setup, press Enter to enter the serial port configuration, and select the option to modify the configuration by entering letters, as shown in the following figure:

```
A - Serial Device      : /dev/ttyUSB0
B - Lockfile Location  : /var/lock
C - Callin Program     :
D - Callout Program    :
E - Bps/Par/Bits       : 115200 8N1
F - Hardware Flow Control : No
G - Software Flow Control : No

Change which setting? █
```

Some notes on configuration options:

1. Serial Device needs to be selected according to the actual device name, usually ttyUSBx.

2. Option E, the specific meaning of configuration is baud rate 115200, 8 data bits, no parity bit and one stop bit.

3. For options F and G, the hardware and software flow control options must be No. After the configuration is complete, press Enter to enter the interface in 1, select Save setup as dfl to save the configuration, and then select Exit from minicom to exit.

4. Use of serial port tools

Enter the command in the terminal:

```
$ sudo minicom
```

4.6.2 Secure programming

Refer to the USBCloner burning tool documentation.pdf

4.6.3 SN burn

Refer to the USBCloner burning tool documentation.pdf

5 FAQ

5.1 How to import the compiled executable program module to the development board?

A: The methods of importing programs include adb, tftp, etc. For specific methods, please refer to <<halley6 Platform Application Development Manual>>.

5.2 Problems compiling the kernel

```
/home/xxx/x1600/kernel/kernel-4.4.94 is not clean, please run 'make mrproper'
in the '/home/xxx/x1600/kernel/kernel-4.4.94' directory.
/home/xxx/x1600/kernel/kernel-4.4.94/Makefile:1003: recipe for target 'prepare3' failed
make[2]: *** [prepare3] Error 1
make[2]: Leaving directory '/home/xxx/x1600/out/product/halley6.v10_nand_4.4.94-eng/obj/kernel-intermediate'
Makefile:150: recipe for target 'sub-make' failed
make[1]: *** [sub-make] Error 2
```

The above phenomenon is usually caused by executing make menuconfig in the kernel directory to configure, and then executing the compile command in the project directory. The solution at this time:

- 1、Execute the command in the kernel directory

```
$make distclean
```

- 2、Go back to the top-level directory of the project to execute lunch to select the board level, and execute the command to compile the kernel

```
$make kernel -j8
```

Tip: All the compilation configurations of this SDK can be compiled and configured in the top-level directory of the project. For details, please refer to the relevant content of this document. It is not recommended to enter a directory for individual configuration.

5.3 The program cannot be executed or the link fails at compile time

Make sure the compiler uses gcc version 7.2.0 (Ingenic Linux-Release5.1.0-Default(xburst1(fp32)+glibc2.29), execute mips-linux-gnu-gcc -v to view:

```
Using built-in specs.
COLLECT_GCC=mips-linux-gnu-gcc
COLLECT_LTO_WRAPPER=/home_bak/jyli/x1600-pure/prebuilts/toolchains/mips-gcc720-glibc2
29/bin/../../libexec/gcc/mips-linux-gnu/7.2.0/lto-wrapper
Target: mips-linux-gnu
Configured with:
/home/users/ylli/ingenic_toolchain/project/src/gcc-7-2017.11/configure
--build=i686-pc-linux-gnu --host=i686-pc-linux-gnu --target=mips-linux-gnu --enable-threads
--disable-libmudflap --disable-libssp --disable-libstdc++-pch --with-arch-32=mips32r2
```

```
--with-arch-64=mips64r2 --with-float=hard --with-mips-plt --enable-extra-sgxxlite-multilibs  
--with-gnu-as --with-gnu-ld
```

```
.....
```

```
urcery/mips-linux-gnu/bin --with-private=default_option_mglibc_mfp32 SED=sed
```

```
Thread model: posix
```

```
gcc version 7.2.0 (Ingenic Linux-Release5.1.0-Default(xburst1(fp32)+glibc2.29)
```

```
2021.12-22 13:09:44)
```

If the above phenomenon does not occur, try to set the compilation environment variable and the lunch board level selection, and refer to the content of the SDK compilation section of this document.